O Problema de Fluxo Máximo em Redes: Implementação do Algoritmo de Ford-Fulkerson em Python

Mário Diego Rocha Valente[1], **Roberto Samarone dos Santos**[2]

[1]Estatístico, Analista de Trânsito(DETRAN-PA)
Graduando em Sistemas de Informação - Universidade Federal do Pará(UFPA)
Rua Augusto Corrêa, 01 - Guamá - Belém - PA - Brasil

[2]Professor Dr., Faculdade de Computação - Universidade Federal do Pará(UFPA)
Rua Augusto Corrêa, 01 - Guamá - Belém - PA - Brasil

mario.vatente@detran.pa.gov.br, rsa@ufpa.br

Abstract. *This paper presents a comprehensive study of the maximum flow problem in networks, focusing on the Ford–Fulkerson algorithm. It covers historical background, formal definitions, the Max-Flow/Min-Cut theorem, classical problems modeled as flows, and practical applications. A step-by-step Python implementation is illustrated with a sample network. For instance, in the first augmenting path* $S \to A \to C \to F \to J \to L \to T$*, the bottleneck capacity was 5, increasing the accumulated flow to 5 units. Following all iterations, the algorithm reaches the optimal maximum flow of* $f = 45$*, in accordance with the Max-Flow/Min-Cut theorem. The main contribution is a didactic presentation that integrates theory and practice, serving as a guide for understanding and applying network flow optimization techniques in computer science, engineering, and operational research contexts.*

Keywords*: Ford–Fulkerson algorithm, maximum flow, network optimization, augmenting paths, bottleneck.*

Resumo. *Este artigo apresenta um estudo detalhado sobre o problema de fluxo máximo em redes, enfatizando o algoritmo de Ford–Fulkerson. São abordados aspectos históricos, definições formais, o Teorema do Fluxo Máximo/Corte Mínimo, problemas clássicos modeláveis por fluxo e suas aplicações práticas. Inclui-se uma implementação na linguagem de programação Python, ilustrada com uma rede exemplo. Na primeira iteração, o caminho aumentante* $S \to A \to C \to F \to J \to L \to T$ *apresentou capacidade mínima de 5, elevando o fluxo acumulado para 5 unidades. Após todas as iterações, o algoritmo atinge o fluxo máximo ótimo* $f = 45$*, conforme previsto pelo Teorema do Fluxo Máximo/Corte Mínimo. A principal contribuição consiste em integrar teoria e prática de forma didática, fornecendo uma referência útil para o ensino e aplicação de técnicas de otimização de fluxo em ciência da computação, engenharia e pesquisa operacional..*

Palavras-chave*: Algoritmo Ford–Fulkerson, fluxo máximo, otimização de redes, caminhos aumentantes, gargalo.*

1. Introdução

A teoria dos grafos consolidou-se como um dos pilares matemáticos e computacionais para a modelagem de problemas de conectividade, transporte e alocação de recursos. Desde o problema das pontes de Königsberg, proposto por Euler no século XVIII [Euler 1736], a ideia de representar entidades como vértices e relações como arestas revelou-se uma abstração poderosa, cuja importância se ampliou ao longo do século XX em áreas como Pesquisa Operacional, Ciência da Computação e Engenharia [Bondy and Murty 1976, West 2001].

Entre as diversas classes de problemas em grafos, destacam-se os *problemas de fluxo em redes*, que modelam o escoamento de recursos de uma fonte s até um sorvedouro t, respeitando restrições de capacidade nos arcos. Essa formulação permite descrever fenômenos de natureza diversa, como transporte de mercadorias, tráfego rodoviário, transmissão de dados, distribuição de energia elétrica e abastecimento de água [Ahuja et al. 1993, **?**]. Em todos esses contextos, a questão central é determinar qual a taxa máxima de transferência possível sem violar as limitações impostas pela rede.

Um marco fundamental nesse campo foi a proposta do método de Ford–Fulkerson, publicada em 1956 [Jr. and Fulkerson 1956]. Essa abordagem introduziu o conceito de *caminhos aumentantes* no *grafo residual* para, de forma iterativa, ampliar o valor do fluxo até alcançar sua máxima capacidade. O método está intimamente relacionado ao *Teorema do Fluxo Máximo–Corte Mínimo*, formalizado por Ford e Fulkerson [Jr. and Fulkerson 1956], que estabelece que o valor máximo de fluxo em uma rede é exatamente igual à capacidade mínima de um corte que separa a fonte do sorvedouro. Essa equivalência teórica não apenas fundamenta o método de Ford–Fulkerson, mas também abriu caminho para uma vasta família de algoritmos posteriores [Cormen et al. 2009].

A partir dessa formulação original, surgiram variações que buscam maior eficiência computacional. O algoritmo de Edmonds–Karp (1972) [Edmonds and Karp 1972] define o uso sistemático da busca em largura (BFS), garantindo complexidade polinomial. O algoritmo de Dinic (1970) [Dinic 1970] introduziu o conceito de grafos de nível e fluxos bloqueadores, ampliando o desempenho em cenários específicos. Posteriormente, técnicas como *push–relabel* (Goldberg–Tarjan, 1986) [Goldberg and Tarjan 1988] exploraram estratégias locais de redistribuição de fluxo, permitindo lidar com redes de grande escala. Essas contribuições consolidaram o problema de fluxo máximo como uma das áreas mais férteis da otimização combinatória.

No contexto contemporâneo, algoritmos de fluxo são aplicados não apenas em sistemas físicos e de transporte, mas também em domínios avançados como segmentação de imagens [Boykov and Jolly 2001], análise de dados em grafos [Ahuja et al. 1993], *routing* em redes de computadores [Kleinberg and Éva Tardos 2006] e planejamento de evacuação em situações de risco [Chen et al. 2010]. Essa versatilidade aponta a relevância do problema e a necessidade de compreender suas bases teóricas e implementações.

Este artigo tem como objetivo apresentar uma análise do problema de fluxo máximo e algoritmo de Ford–Fulkerson. Serão discutidos seus fundamentos conceituais, pseudocódigo e variações, limitações e um estudo de caso com implementação prática, a fim de apresentar seu papel como ferramenta de modelagem e otimização.

2. Trabalhos Correlatos

A teoria de fluxo em redes e o algoritmo de Ford-Fulkerson constituem pilares fundamentais na pesquisa operacional e na ciência da computação, sendo amplamente estudados desde a década de 1950. A seguir, apresenta-se uma visão histórica e evolutiva dos principais trabalhos correlatos, suas contribuições e variações do algoritmo.

O algoritmo de Ford-Fulkerson foi proposto por **L.R. Ford Jr. e D.R. Fulkerson** em 1956, no artigo seminal [Jr. and Fulkerson 1956]. O método introduziu conceitos fundamentais, como rede residual, caminho aumentante e corte mínimo, consolidando o problema do fluxo máximo como uma área central na otimização de redes e em problemas como emparelhamento em grafos bipartidos.

Em 1972, **Jack Edmonds e Richard Karp** apresentaram uma melhoria significativa, escolhendo sempre o caminho aumentante mais curto (em número de arestas) por meio de uma busca em largura (BFS) [Edmonds and Karp 1972]. Essa implementação, conhecida como *Algoritmo de Edmonds-Karp*, garante convergência em tempo polinomial, com complexidade $O(V \cdot E^2)$, evitando ciclos infinitos que poderiam ocorrer no algoritmo original.

Na década de 1970, **Yefim Dinic** desenvolveu o conceito de grafo de nível e fluxos bloqueadores [Dinic 1970], permitindo encontrar múltiplos caminhos aumentantes simultaneamente. O algoritmo de *Dinic* tem complexidade $O(V^2 \cdot E)$ no caso geral e $O(\min(V^{2/3}, E^{1/2}) \cdot E)$ em grafos unitários, representando um avanço no desempenho do algoritmo original.

Em 1986, **Goldberg e Tarjan** propuseram o método *Push-Relabel* [Goldberg and Tarjan 1988], baseado em pré-fluxos, que violam temporariamente a conservação do fluxo e são ajustados localmente por operações de *push* e *relabel*. Essa abordagem permite redistribuir fluxo sem depender de caminhos aumentantes globais, alcançando complexidade $O(V^3)$ no caso geral, com refinamentos que reduzem para $O(V^2\sqrt{E})$.

Ahuja e Orlin (1992) propuseram o *Capacity Scaling* [Ahuja et al. 1993], otimizando Ford-Fulkerson ao considerar apenas caminhos com capacidade acima de um limite, reduzindo buscas inúteis. Paralelamente, surgiram algoritmos distribuídos e paralelos, voltados a redes de computadores e processamento de grafos massivos, consolidando a aplicabilidade prática da teoria de fluxo em larga escala.

James Orlin (2013) anunciou o algoritmo de fluxo máximo mais rápido conhecido até então [Orlin 2013], com complexidade $O(VE)$. **Goldberg e Rao** desenvolveram o algoritmo assintoticamente mais rápido para fluxo máximo, baseado em fluxos de bloqueio [Goldberg and Rao 1998], com tempo de execução $O(\min(V^{2/3}, E^{1/2}) \cdot E \cdot \log(V^2/E + 2) \cdot \log C)$, sem utilizar push-relabel.

3. Redes de Fluxo (ou Rede Capacitada)

As redes de fluxo constituem um modelo matemático fundamental em teoria dos grafos, aplicado na resolução de problemas de transporte, logística e comunicação [Ahuja et al. 1993, Cormen et al. 2009].

Uma rede de fluxo é um grafo direcionado, $G = (V, E)$, onde:

- V é o conjunto de vértices (nós) da rede.
- E é o conjunto de arestas (arcos) direcionadas que conectam os vértices.

A cada aresta $(u, v) \in E$ está associado um número real não negativo $c(u, v)$, denominado capacidade da aresta. A capacidade $c(u, v)$ representa a quantidade máxima de fluxo que pode passar de u para v por unidade de tempo. Se não houver aresta de u para v, convencionalmente $c(u, v) = 0$.

Existem dois vértices especiais: uma fonte (s), que é o ponto de origem do fluxo, e um sumidouro (t), que é o ponto de destino ou consumo do fluxo. Todos os outros vértices são intermediários e apenas propagam o fluxo.

3.1. Fluxo

Um fluxo em uma rede de fluxo G é uma função $f : V \times V \to \mathbb{R}$ (que associa um número real a cada par de vértices) que satisfaz as seguintes propriedades [Ahuja et al. 1993],:

- **Restrição de Capacidade**: Para cada par de vértices $(u, v) \in V \times V$, o fluxo $f(u, v)$ deve ser não negativo e não pode exceder a capacidade da aresta: $0 \leq f(u, v) \leq c(u, v)$ Se não houver aresta (u, v) no grafo original, então $f(u, v) = 0$;
- Conservação do Fluxo: Para cada vértice $u \in V$, exceto a fonte s e o sumidouro t, o fluxo total que entra em u deve ser igual ao fluxo total que sai de u: $\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$ para todo $u \in V \setminus s, t$ Isso significa que não há acúmulo nem perda de material nos vértices intermediários;
- **Valor do Fluxo**: O valor de um fluxo f, denotado por $|f|$, é o fluxo líquido total que sai da fonte s: $|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s)$ Normalmente, em uma rede de fluxo, não há arestas de entrada na fonte, então o segundo termo é zero. O objetivo do problema de fluxo máximo é encontrar um fluxo f tal que seu valor $|f|$ seja o maior possível;
- **Redes com Múltiplas Fontes e Sumidouros**: O problema de fluxo máximo pode ser estendido para redes com múltiplas fontes e múltiplos sumidouros [Goldberg and Tarjan 1988, Ahuja et al. 1993].Para resolver esses casos, a rede original pode ser transformada em uma rede equivalente com uma única fonte e um único sumidouro:
  - Superfonte (s'): É adicionada uma nova superfonte s' e arestas direcionadas de s' para cada fonte original s_i com capacidade infinita $c(s', s_i) = \infty$;
  - Supersumidouro (t'): É adicionado um novo supersumidouro t' e arestas direcionadas de cada sumidouro original t_j para t' com capacidade infinita $c(t_j, t') = \infty$. Qualquer fluxo na rede original corresponde a um fluxo na rede transformada, e vice-versa.
- **Capacidades em Vértices**: Em algumas redes, os vértices também podem ter capacidades que limitam a quantidade de fluxo que pode passar por eles. Esse problema também pode ser transformado em uma rede equivalente onde todas as

capacidades estão nas arestas. Isso é feito dividindo cada vértice v com capacidade $l(v)$ em dois vértices, v_{in} e v_{out}, conectados por uma aresta (v_{in}, v_{out}) com capacidade $c(v_{in}, v_{out}) = l(v)$. Todas as arestas de entrada para v agora vão para v_{in}, e todas as arestas de saída de v agora saem de v_{out}.

O problema de fluxo máximo, portanto, busca o valor máximo de fluxo possível, respeitando todas as capacidades das arestas e as leis de conservação de fluxo, tornando-o uma ferramenta fundamental para otimizar a passagem de recursos através de qualquer sistema que possa ser modelado como uma rede.

4. Teorema do Fluxo Máximo/Corte Mínimo

Um dos resultados mais importantes da teoria de redes de fluxo é o Teorema do Fluxo Máximo, formulado por Ford e Fulkerson [**?**].

Para entender este teorema, precisamos primeiro definir o conceito de um corte em uma rede de fluxo. Um corte (S, T) em uma rede de fluxo $G = (V, E)$ com fonte s e sumidouro t é uma partição do conjunto de vértices V em dois subconjuntos S e $T = V \setminus S$, tal que a fonte $s \in S$ e o sumidouro $t \in T$.

A capacidade de um corte $c(S, T)$ é a soma das capacidades de todas as arestas que vão de um vértice em S para um vértice em T: $c(S, T) = \sum_{(u,v) \in E com u \in S, v \in T} c(u, v)$ Note que arestas de T para S não contribuem para a capacidade do corte neste sentido, pois o fluxo é unidirecional de S para T.

O Teorema do Fluxo Máximo/Corte Mínimo [Cormen et al. 2009], estabelece que o valor do fluxo máximo em uma rede é igual à capacidade de um corte mínimo na mesma rede. Um corte mínimo é um corte cuja capacidade é a menor entre todos os cortes possíveis na rede.

Este teorema é central porque fornece uma condição de otimalidade para o fluxo encontrado. Seja f um fluxo em uma rede G com fonte s e sumidouro t. As seguintes condições são equivalentes:

1. f é um fluxo máximo em G.
2. A rede residual G_f não contém nenhum caminho aumentante.
3. $|f| = c(S, T)$ para algum corte (S, T) de G.

A equivalência dessas condições é crucial: se um algoritmo encontra um fluxo f e não consegue mais encontrar caminhos aumentantes na rede residual G_f, então ele encontrou um fluxo máximo. Além disso, a capacidade desse fluxo máximo será exatamente igual à capacidade do corte mínimo da rede, que é o ”gargalo”ou a ”vulnerabilidade”da rede que limita o fluxo total.

A prova formal deste teorema é um processo de indução que demonstra que, para qualquer fluxo, o valor do fluxo é sempre menor ou igual à capacidade de qualquer corte, e que para o fluxo máximo existe um corte cuja capacidade é exatamente igual ao valor do fluxo.

4.1. Algoritmo de Ford–Fulkerson (pseudocódigo)

seguir apresentamos um pseudocódigo em estilo compatível com a literatura clássica, porém redigido neste trabalho [Cormen et al. 2009, Ahuja et al. 1993, Kleinberg and Éva Tardos 2006].

A estratégia geral é iterar enquanto existir caminho aumentante entre s e t no grafo residual, atualizando os fluxos até que não haja mais possibilidade de incremento.

Algorithm 1 Algoritmo de Ford–Fulkerson

```
procedure FORDFULKERSON(G = (V, E), s, t)
    Inicialize f(u, v) ← 0 para toda aresta (u, v) ∈ E
    while existe caminho aumentante P de s até t na rede residual G_f do
        Construir o grafo residual G_f a partir do fluxo f
        Encontrar um caminho aumentante P de s até t em G_f (ex.: DFS ou BFS)
        Δ ← min{c_f(u, v) | (u, v) ∈ P}          ▷ Capacidade residual mínima de P
        for cada aresta (u, v) em P do
            f(u, v) ← f(u, v) + Δ                ▷ Aumenta fluxo na direção direta
            f(v, u) ← f(v, u) − Δ   ▷ Permite fluxo reverso (desfazer se necessário)
        end for
    end while
    return f                                     ▷ Fluxo máximo encontrado
end procedure
```

Se a busca por caminhos aumentantes for feita por BFS (sempre escolhendo o caminho mais curto em número de arestas), obtém-se a variante *Edmonds–Karp*, cuja complexidade é $\mathcal{O}(|V||E|^2)$ [Edmonds and Karp 1972, Ahuja et al. 1993].

Com uma estratégia arbitrária (DFS), o número de iterações pode depender do valor do fluxo máximo quando as capacidades são inteiras [Jr. and Fulkerson 1956, Tarjan 1983].

4.2. Problemas clássicos Modeláveis como Fluxo

Diversos problemas que podem ser representados e resolvidos com problemas de fluxo de redes [Ahuja et al. 1993, Cormen et al. 2009]. Entre os principais, destacam-se:

- Fluxo Máximo em Redes de Transporte ou Logística;
- Problemas de Corte Mínimo em Redes;
- Problemas de Emparelahamento em Grafos Bipartidos;
- Distribuição de Recursos ou Tarefas em Sistemas Industriais

4.3. Aplicações Práticas

O problema de fluxo máximo e suas variações têm ampla aplicação em diversas áreas:

- Transporte de Àgua, Energia Elétrica ou Dados em Redes;
- Planejamento de Tráfego ou Logística de Transporte de Mercadorias;
- Alocação de Tarefas em Computadores ou Projetos;
- Redes Sociais e Análise de Influência.

Essa trajetória histórica demonstra como o algoritmo de Ford-Fulkerson evoluiu e se adaptou, consolidando-se como um pilar teórico e prático na ciência da computação e em aplicações de engenharia e logística.

5. Vantagens e Desvantagens

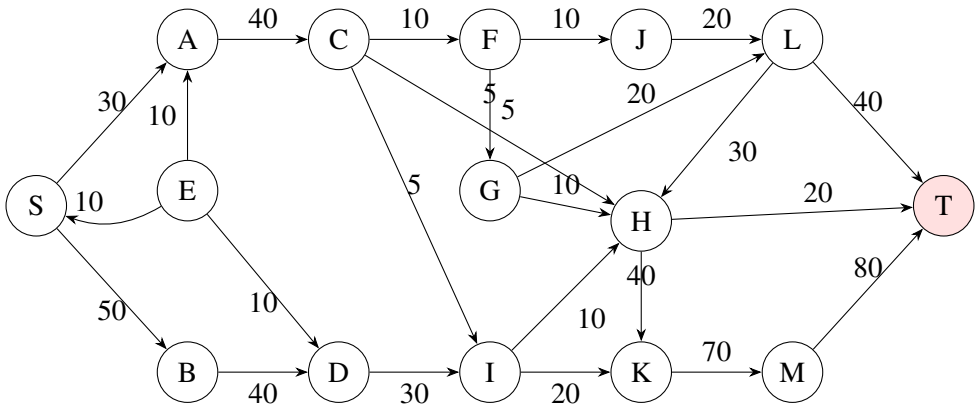
- **Vantagens**: eficiência computacional (algoritmos polinomiais), simplicidade do modelo, aplicabilidade ampla.
- **Desvantagens**: dificuldade de modelar restrições complexas, não captura todos os aspectos de problemas dinâmicos, dependência de dados precisos.

6. Resultados

6.1. Exemplo Prático

Nesta seção, ilustramos o funcionamento do Ford–Fulkerson em uma rede clássica. Usaremos os vértices $S, A, B, C, D, E, F, G, H, I, J, K, L, M, T$ e as capacidades mostradas na Figura,1. O objetivo é determinar o fluxo máximo de S para T.

Figura 1. Rede de teste para aplicação do algoritmo de Ford–Fulkerson



6.2. Implementação em Python

A implementação do algoritmo de Ford-Fulkerson em Python, particularmente a versão Edmonds-Karp, envolve representar o grafo e a lógica para encontrar caminhos aumentantes e atualizar o fluxo.

```python
#---------------------- Pacotes ------------------------------#
from collections import deque, defaultdict
#-------------------- Definicao do Grafo --------------------#
graph = {
    'S': {'A': 30, 'B': 50},
    'A': {'C': 40},
    'B': {'D': 40},
    'C': {'F': 10, 'H': 5, 'I':5},
    'D': {'I': 30},
    'F': {'G': 5, 'J': 10},
    'G': {'H': 20, 'L': 20},
    'H': {'K': 40, 'T': 20},
    'I': {'H': 10, 'K': 20},
    'J': {'L': 20},
    'K': {'M': 70},
    'L': {'H':30 ,'T': 40},
    'M': {'T': 80},
    'T': {}
    }
#-------------------------------------------------------------#
```

```python
# -------------- Algoritmo de Ford-Fulkerson ---------------- #
def bfs(rGraph, source, sink, parent):
    visited = set()
    queue = deque([source])
    visited.add(source)
    while queue:
        u = queue.popleft()
        for v, cap in rGraph[u].items():
            if v not in visited and cap > 0:
                parent[v] = u
                visited.add(v)
                queue.append(v)
                if v == sink:
                    return True
    return False
def ford_fulkerson(graph, source, sink):
```

```python
#------------------- Criar Grafo Residual --------------------#
    rGraph = defaultdict(dict)
    for u in graph:
        for v, cap in graph[u].items():
            rGraph[u][v] = cap
            rGraph[v][u] = 0  # Aresta reversa
    max_flow = 0
    paths = []  # Para armazenar caminhos aumentantes
    parent = {}
    while bfs(rGraph, source, sink, parent):
#-------------------------------------------------------------#
```

```python
#-------- Encontrar Fluxo Minimo no Caminho Aumentante -------#
        path_flow = float("Inf")
        v = sink
        path = []
        while v != source:
            u = parent[v]
            path_flow = min(path_flow, rGraph[u][v])
            path.append(v)
            v = u
        path.append(source)
        path.reverse()
#-------------------------------------------------------------#
```

```python
#----------------- Atualizar Grafo Residual ------------------#
        v = sink
        while v != source:
            u = parent[v]
            rGraph[u][v] -= path_flow
            rGraph[v][u] += path_flow
            v = u
        max_flow += path_flow
        paths.append((path, path_flow))  # Armazenar caminho +
           fluxo
        parent = {}  # Resetar parent para proxima iteracao
    return max_flow, paths
#-------------------------------------------------------------#
```

```python
# ------------------------ Execucao -------------------------- #
if __name__ == "__main__":
    max_flow, paths = ford_fulkerson(graph, 'S', 'T')

    print("Fluxo Maximo =", max_flow)
    print("\n Caminhos Aumentantes:")
    for i, (p, f) in enumerate(paths, 1):
        print(f"{i}: Caminho {p}  |     = {f}")
#-------------------------------------------------------------#
```

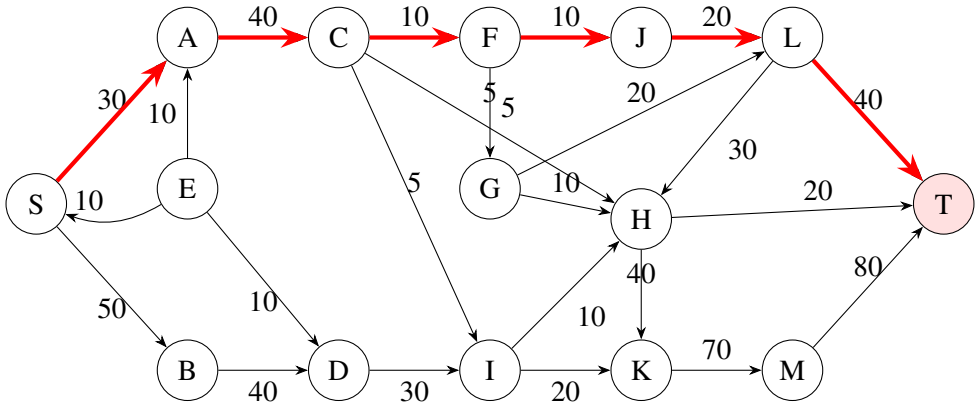
6.3. Aplicação do Algoritmo Ford–Fulkerson

A seguir aplicamos o Ford–Fulkerson à rede da Fig. 1. Em cada iteração escolhemos um caminho aumentante, calculamos o gargalo Δ e atualizamos o valor acumulado do fluxo $|f|$. Para clareza, em cada figura o caminho escolhido é destacado em **vermnelho**.

Iteração 1. Caminho aumentante

$$S \to A \to C \to F \to J \to L \to T$$

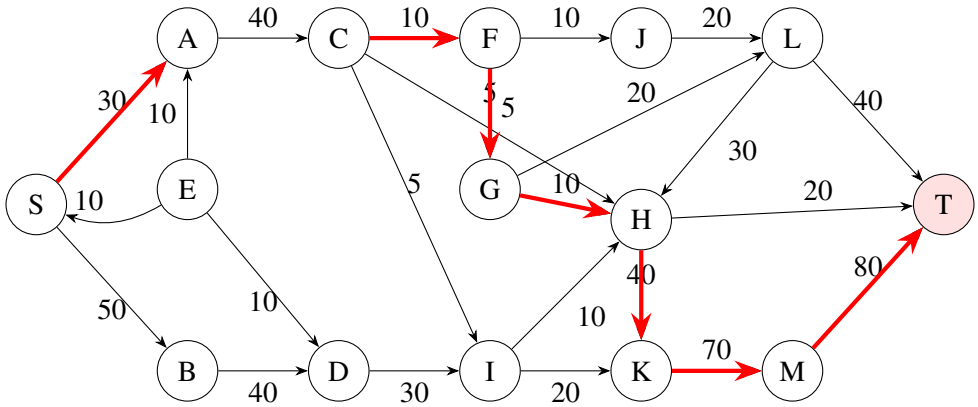
Capacidades ao longo do caminho: $(30, 40, 10, 10, 20, 40)$, portanto $\Delta = \min = \mathbf{5}$. Fluxo acumulado após a iteração: $|f| = 0 + 5 = \mathbf{5}$.



Iteração 2. Caminho aumentante

$$S \to A \to C \to F \to G \to H \to K \to M \to T$$

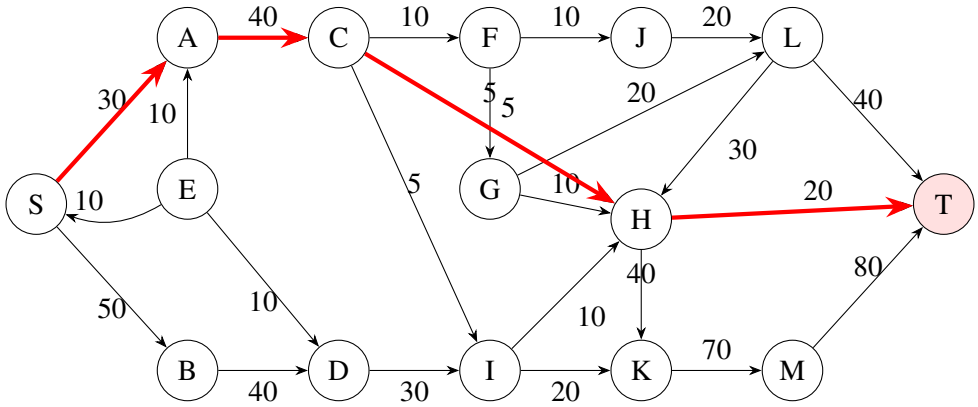
Capacidades: $(30, 40, 10, 5, 10, 40, 70, 80) \Rightarrow \Delta = \mathbf{5}$. Atualização: $|f| \leftarrow 5 + 5 = \mathbf{10}$.



Iteração 3. Caminho aumentante

$$S \to A \to C \to H \to T$$

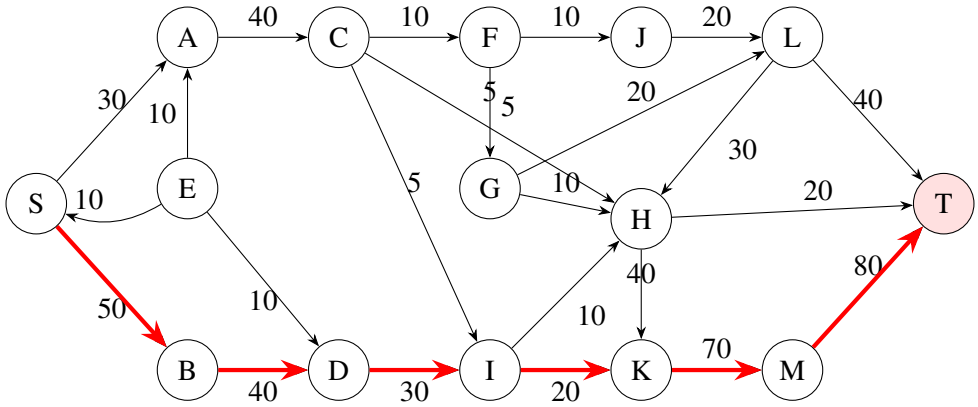
Capacidades: $(30, 40, 5, 20) \Rightarrow \Delta = \mathbf{5}$. Atualização: $|f| \leftarrow 10 + 5 = \mathbf{15}$.



Iteração 4. Caminho aumentante

$$S \to B \to D \to I \to K \to M \to T$$

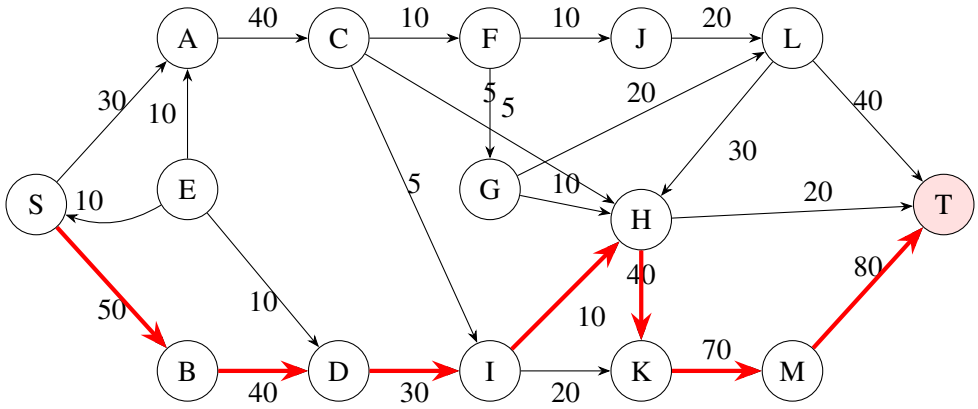
Capacidades: $(50, 40, 30, 20, 70, 80) \Rightarrow \Delta = \mathbf{20}$. Atualização: $|f| \leftarrow 15 + 20 = \mathbf{35}$.



Iteração 5. Caminho aumentante

$$S \to B \to D \to I \to H \to K \to M \to T$$

Capacidades (após as iterações anteriores): $(50,\ 40,\ 10 - -rem,\ 10,\ 40,\ 70,\ 80) \Rightarrow \Delta = \mathbf{10}. Atualização: |f| \leftarrow 35 + 10 = \mathbf{45}$.



Ótimo (parada). Não há mais caminho aumentante que aumente $|f|$ além de 45. De fato, o corte

$$S^* = \{S, A, B, C, D, I\}, \quad T^* = V \setminus S^*$$

tem capacidade

$$c(S^*, T^*) = c(C, F) + c(C, H) + c(I, K) + c(I, H) = 10 + 5 + 20 + 10 = \mathbf{45},$$

logo, pelo Teorema Fluxo Máximo–Corte Mínimo, o valor ótimo é $|f| = \mathbf{45}$.

Tabela 1. Resumo das iterações do algoritmo Ford–Fulkerson na rede da Fig. 1.

Iteração	Caminho Aumentante	Δ	$\lvert f \rvert$ acumulado
1	$S \to A \to C \to F \to J \to L \to T$	5	5
2	$S \to A \to C \to F \to G \to H \to K \to M \to T$	5	10
3	$S \to A \to C \to H \to T$	5	15
4	$S \to B \to D \to I \to K \to M \to T$	20	35
5	$S \to B \to D \to I \to H \to K \to M \to T$	10	45

A aplicação do algoritmo de Edmonds–Karp à rede definida na Fig. 1 resultou na identificação de diversos caminhos aumentantes, com seus respectivos gargalos (fluxos possíveis em cada iteração) e atualização do fluxo acumulado f. A cada iteração, o algoritmo utilizou uma busca em largura (BFS) para encontrar o caminho mais curto em número de arestas entre a fonte S e o sorvedouro T. Em seguida, o gargalo Δ foi calculado como a menor capacidade residual ao longo do caminho. Por fim, as capacidades residuais foram atualizadas e o valor de f ajustado.

7. Considerações Finais

O problema de fluxo máximo em redes, juntamente com o método de Ford-Fulkerson, representa uma área de estudo fundamental e de imensa relevância prática na otimização combinatória e na ciência da computação [Jr. and Fulkerson 1956]. Como demonstrado ao longo deste artigo, a capacidade de modelar sistemas complexos de transporte, distribuição e comunicação como redes de fluxo permite resolver uma variedade impressionante de problemas do mundo real [Ahuja et al. 1993].

A metodologia de Ford-Fulkerson, embora concebida há décadas por L.R. Ford Jr. e D.R. Fulkerson, continua sendo a base para a compreensão e o desenvolvimento de algoritmos mais avançados. Seus conceitos centrais de redes residuais, caminhos aumentantes e cortes são indispensáveis para analisar a dinâmica de fluxo e identificar os gargalos (cortes mínimos) que limitam a capacidade de uma rede [Cormen et al. 2009].

A versão Edmonds-Karp, que emprega a busca em largura para encontrar caminhos aumentantes, garante um tempo de execução polinomial, tornando-o uma escolha robusta para muitas aplicações [**?**]. No entanto, é crucial reconhecer tanto as vantagens, como a versatilidade e a capacidade de fornecer soluções exatas e identificar vulnerabilidades, quanto as desvantagens, como a potencial ineficiência em sua forma genérica e a sensibilidade a capacidades irracionais. A contínua pesquisa na área, exemplificada pelos algoritmos push-relabel e pelas contribuições de autores como Goldberg e Tarjan, reflete a busca por soluções ainda mais rápidas e eficientes para lidar com redes de grande escala e complexidade [Goldberg and Tarjan 1988, **?**].

A implementação conceitual em Python ilustra como a teoria abstrata pode ser traduzida em código, fornecendo uma ferramenta prática para a análise de redes. A clareza e a flexibilidade do Python facilitam a construção e manipulação das estruturas de dados necessárias, tornando o algoritmo acessível para experimentação e aplicação em diversos contextos [Van Rossum and Drake 2009].

Em suma, o problema de fluxo máximo e o algoritmo de Ford-Fulkerson são mais do que apenas um exercício acadêmico; são ferramentas poderosas que capacitam engenheiros, cientistas de dados e pesquisadores a otimizar sistemas, alocar recursos de forma inteligente e tomar decisões estratégicas, contribuindo significativamente para a eficiência e resiliência de infraestruturas modernas [Kleinberg and Éva Tardos 2006]. A compreensão de seus princípios e a capacidade de aplicá-los continuam sendo habilidades valiosas no cenário tecnológico atual.

8. Referências Bibliográficas

Referências

[Ahuja et al. 1993] Ahuja, R. K., Magnanti, T. L., and Orlin, J. B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall.

[Bondy and Murty 1976] Bondy, J. A. and Murty, U. S. R. (1976). *Graph Theory with Applications*. Macmillan.

[Boykov and Jolly 2001] Boykov, Y. and Jolly, M.-P. (2001). Interactive graph cuts for optimal boundary and region segmentation of objects in n-d images. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 105–112.

[Chen et al. 2010] Chen, J., Deogun, J., and Wu, T. (2010). Flow-based evacuation planning. In *Proceedings of the IEEE International Conference on Technologies for Homeland Security*, pages 345–350.

[Cormen et al. 2009] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*. MIT Press, 3rd edition.

[Dinic 1970] Dinic, Y. A. (1970). Algorithm for solution of a problem of maximum flow in a network with power estimation. *Soviet Math. Dokl.*, 11:1277–1280.

[Edmonds and Karp 1972] Edmonds, J. and Karp, R. M. (1972). Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264.

[Euler 1736] Euler, L. (1736). Solutio problematis ad geometriam situs pertinentis. *Commentarii academiae scientiarum Petropolitanae*, 8:128–140.

[Goldberg and Rao 1998] Goldberg, A. V. and Rao, S. (1998). Faster algorithms for the maximum flow problem. *Journal of Algorithms*, 25(1):1–29.

[Goldberg and Tarjan 1988] Goldberg, A. V. and Tarjan, R. E. (1988). A new approach to the maximum flow problem. *Journal of the ACM*, 35(4):921–940.

[Jr. and Fulkerson 1956] Jr., L. R. F. and Fulkerson, D. R. (1956). Maximal flow through a network. *Canadian Journal of Mathematics*, 8(3):399–404.

[Kleinberg and Éva Tardos 2006] Kleinberg, J. and Éva Tardos (2006). *Algorithm Design*. Pearson.

[Orlin 2013] Orlin, J. B. (2013). Max flows in o(nm) time, or better. *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, pages 765–774.

[Tarjan 1983] Tarjan, R. E. (1983). *Data Structures and Network Algorithms*. Society for Industrial and Applied Mathematics (SIAM), Philadelphia, PA.

[Van Rossum and Drake 2009] Van Rossum, G. and Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace, Scotts Valley, CA.

[West 2001] West, D. B. (2001). *Introduction to Graph Theory*. Prentice Hall, 2nd edition.